1 Lesson 29-10-26

1.1 Synchronous protocols

In the last lesson we saw serial protocols, more specifically UART. UART uses separate clocks at RX and TX, with synchronization on the START bit.

We know want to see protocols, either character or bit oriented, where clock recovery uses transitions of the data signal, in each bit (synchronous carrier), so periodically.

First some constraints: The *channel bandwidth* requirement is that it contains at least the highest fundamental frequency in the transmitted signal. The lower, the better, given a certain Bit Rate f_{ck}, as it makes for a cheaper channel. This means we need to find the worst case depending on the bits transmitted.

For the number of *voltage levels* used in the channel, we stick with standard logic which uses 2: $0\,\text{V}$ and V_{dd}.

1.1.1 NRZ encoding

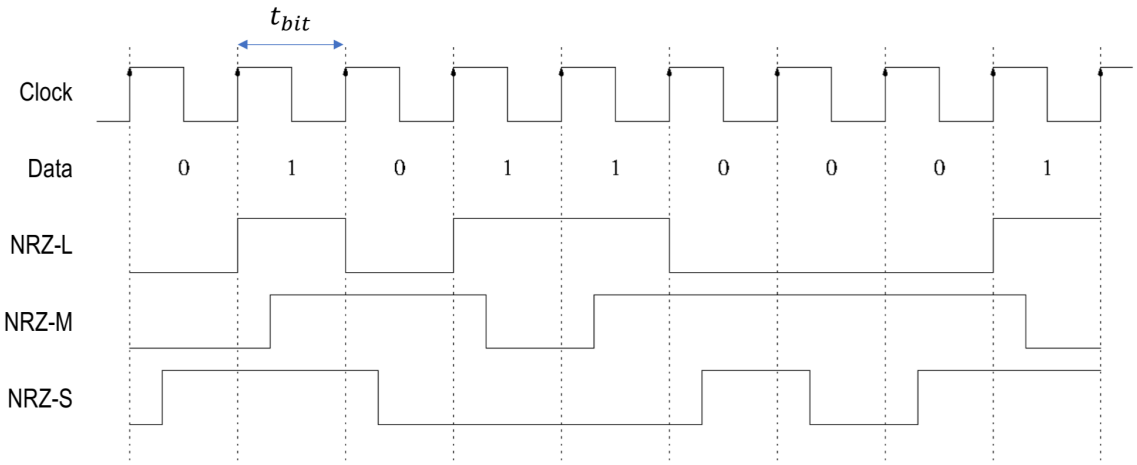
A first approach is **NRZ** (*Non Return to Zero Encoding*). Several types:

- **NRZ-L** (*Level*):
  - Logic 1: high state
  - Logic 0: low state
- **NRZ-M** (*Mark*, also known as **NRZ-I**, *Inverted*):
  - Logic 1: represented by a transition
  - Logic 0: no change
- **NRZ-S** (*Space*, also known as **NRZ-I**, *Inverted*):
  - Logic 1: no change
  - Logic 0: represented by a transition

Note that in none of these transitions are *guaranteed*! We might not have them, which prevents us from synchronizing on that clock cycle.

Also, since at most we change value once per clock, the maximum frequency for NRZ is:

$$f_{\text{max}} = \frac{f_{\text{ck}}}{2}$$



1.1.2 RZ encoding

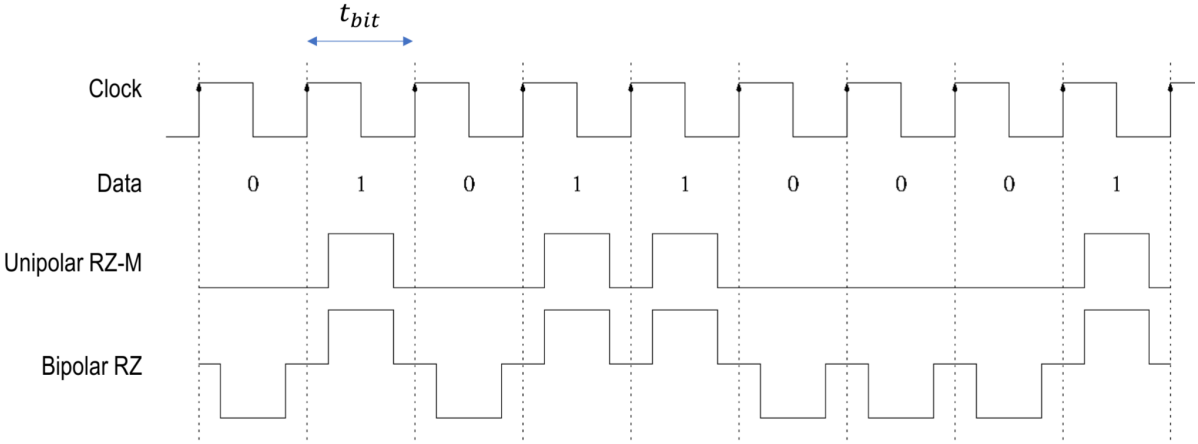
The alternative is **RZ** (*Return to Zero Encoding*). Several types of this too:

- Unipolar **RZ-M**
  - Logic 1: high pulse
  - Logic 0: low state
- Bipolar **RZ**
  - Logic 1: positive pulse
  - Logic 0: negative pulse

Transitions are not guaranteed in unipolar, but they are in bipolar (good!), tough it needs a ternary code with better SNR.

For the bandwidth, we can change the value two times per clock, so the maximum frequency for both RZ types is:

$$f_{\max} = f_{\text{ck}}$$



An alternative that exists is **AMI** RZ (*Alternative Mark Inverted Return to Zero*). This involves marking:

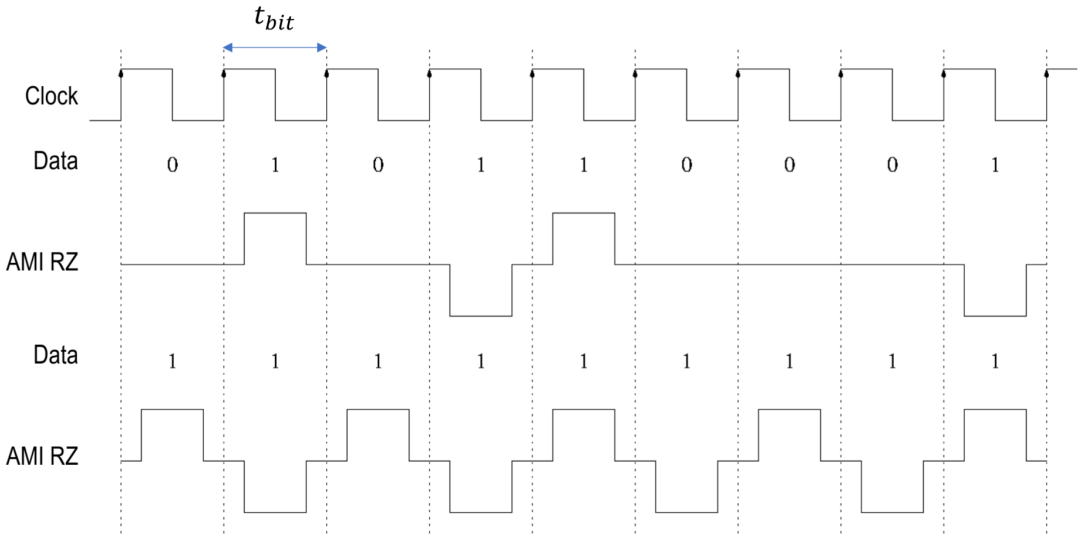
- Logic 1: alternate level pulse
- Logic 0: no pulse

Where, by alternate level pulse we mean a pulse opposite of the previous one. We consider this because we find that the maximum frequency is in fact reduced:

$$f_{\max} = \frac{f_{\text{ck}}}{2}$$

This is because the worst case scenario, where we transmit a high value at each cycle, gets us basically a sinusoidal wave at frequency $\frac{f_{\text{ck}}}{2}$. Also, the DC component is absent on average, which is an advantage in long connections.

The tradeoff is that this clearly doesn't guarantee transitions.



1.1.3 DICODE and DUOBIN encoding

These are based on converting input transitions (in the data to transmit) to output pulses.

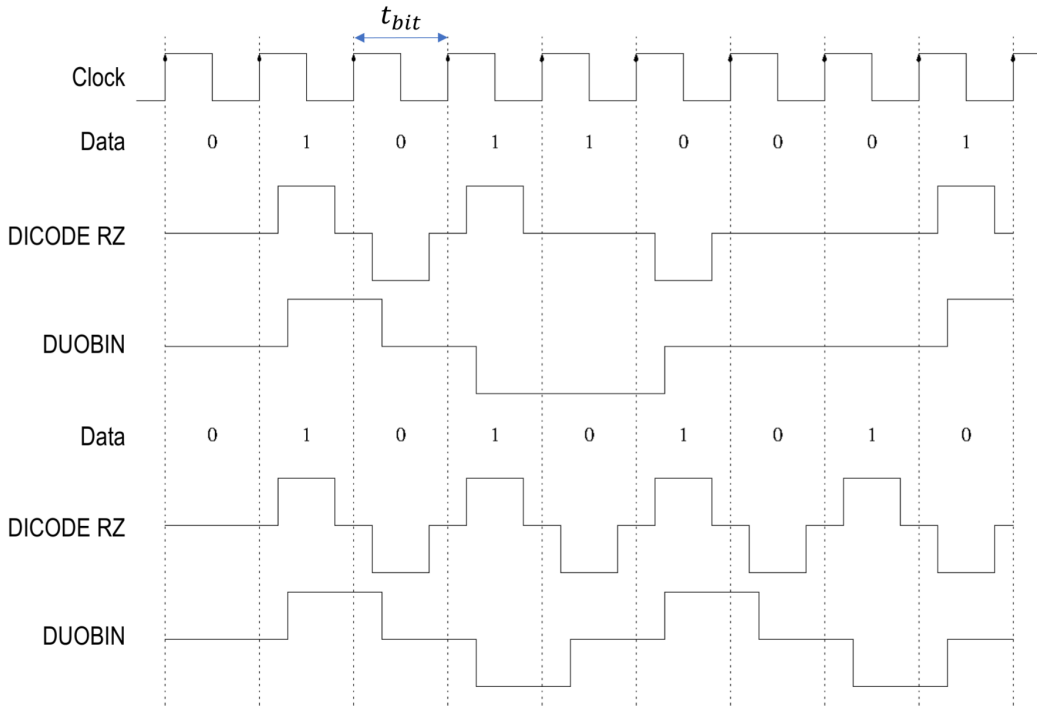
- **DICODE RZ**: each input transition generates an output pulse, with alternate polarity. If there is no input transition, the output remains stable;
- **DUOBIN** (*DICODE NRZ*): each input transition generates an output transition, with alternate polarity. If there is no input transition, the output remains stable.

These clearly don't guarantee transistions, but are really efficient in terms of bandwidth as:

$$f_{\max} = \frac{f_{\text{ck}}}{2} \quad \text{(DICODE)}$$

$$f_{\max} = \frac{f_{\text{ck}}}{4} \quad \text{(DUOBIN)}$$

in the best case.



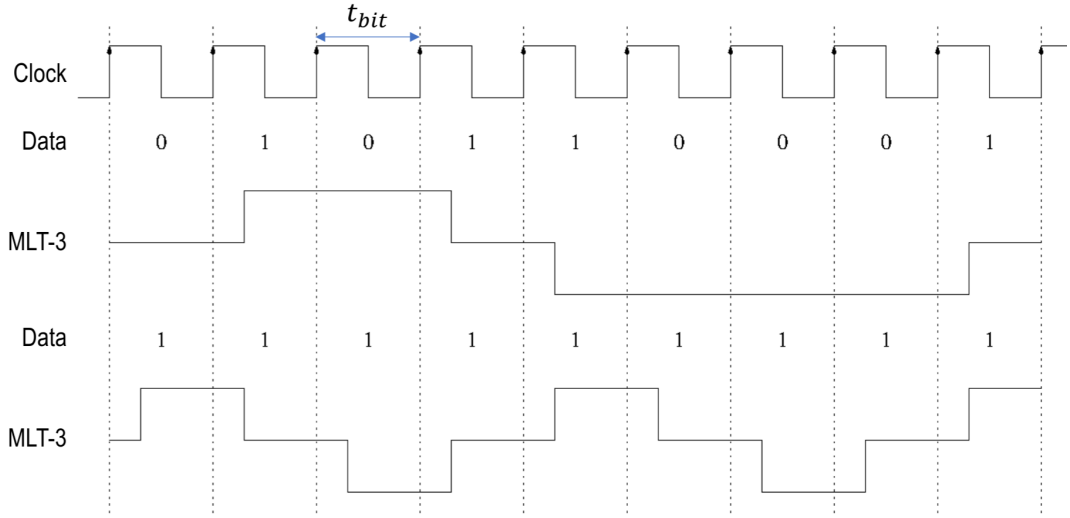
1.1.4 MLT encoding

This is significant because it used in Ethernet 100Base-TX (I would guess one of the thicknet standards). For instance, MLT-3 has three logic levels:

- Logic 0: no change
- Logic 1: output transition with alternate polarity

The bandiwdth is good as we get:

$$f_{\max} = \frac{f_{\text{ck}}}{4}$$



1.1.5 Manchester codes

Also known as IEEE 802.3, **manchester codes** encode data via only transitions. They can be obtained from NRZ-L (see 16.1.1) by XOR'ing with the clock.

- Logic 1: low to high transition (L $\rightarrow$ H)
- Logic 0: high to low transition (H $\rightarrow$ L)

For the opposite encoding:

- Logic 1: high to low transition (H $\rightarrow$ L)
- Logic 0: low to high transition (L $\rightarrow$ H)

Transitions are guaranteed (which is good), but bandwidth of course suffers:

$$f_{\text{max}} \leq f_{\text{ck}}$$

There are some variations.

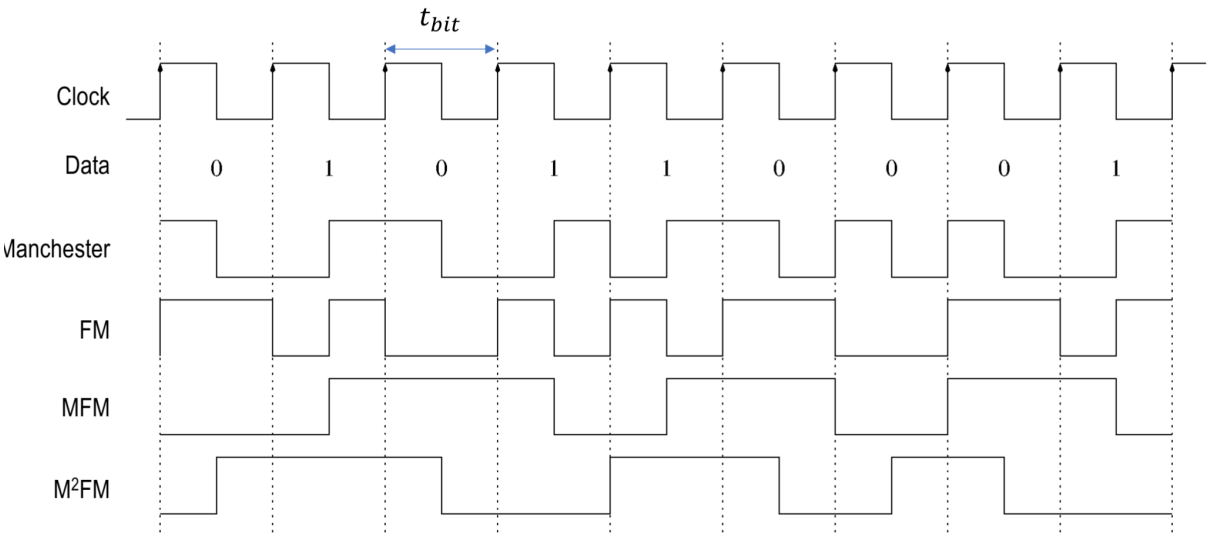
- First off, **FM** (*Frequency Modulation*). Also known as BiPhase or Differential Manchester Encoding:
  - Logic 1: a transition at the beginning of the bit, followed by a transition in the middle of the bit
  - Logic 0: a transition at the beginning of the bit
- **MFM** is *Modified Frequency Modulation*. The idea is to have different transition periods based on the value transmitted:
  - Logic 1: a transition in the middle of the bit
  - Logic 0: no transition if preceded by a 1, a transition at the beginning if preceded by a 0
- Lastly we have **M2MF**, which is somehow named *Modified Modified Frequency Modulation*. In particular, a clock pulse is only inserted between a pair of adjacent 0-bits if the first bit of the pair did not have a clock pulse inserted before it.

  This is wildly non standard as it seems the professor's slides interpret it differently (?).

All of these guarantees transitions.

For bandiwdth, as we said, manchester is f_{ck}. FM is also f_{ck}, whereas MFM and M2FM are:

$$f_{\text{max}} = \frac{f_{\text{ck}}}{2}$$



1.1.6 Forcing transitions

Some encodings show excellent characteristics, such as simplicity, low bandwidth, two voltage levels, but do not guarantee transitions.

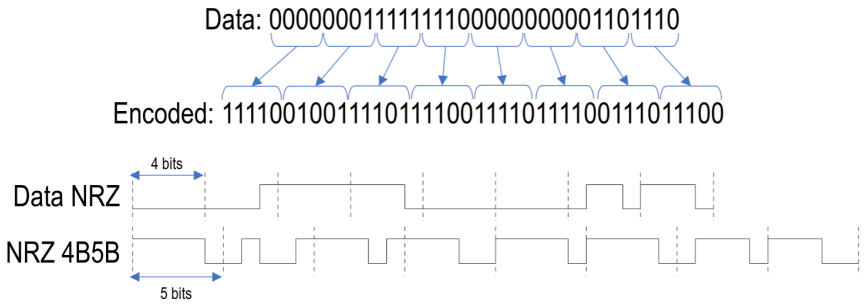
We can solve this by forcing transitions into our data before encoding:

- **xByB**: every x-bit block is replaced by a y-bit block, with $y > x$. Because of how the y blocks are constructed this imposes the presence of transitions, is deterministic, but imposes a fixed overhead.

  Redundancy is used for transition insertions, but not at every bit: instead, they are guaranteed every n bits. xByB can also be used for DC and low frequencies removal.

4B	5B
0000	11110
0001	01001
0010	10100
0011	10101
0100	01010
0101	01011
0110	01110
0111	01111

4B	5B
1000	10010
1001	10011
1010	10110
1011	10111
1100	11010
1101	11011
1110	11100
1111	11101



There are a number of xByB codes, such as 3B4B, 4B5B, ecc... with different characteristics. They can all easily be implemented via LUTs.

- **Bit stuffing** involves, every n bits that don't cause a transition, to insert a bit that causes a transition.

  The value of n is normally between 5 and 8, as the number of acceptable consecutive bits without a transition determined using the eye diagram. It differs depending on the protocol, with transmitter and receiver clocks re-synchronized at transitions.

  Of course at the receiver we have to notice if a bit that causes a transition is an actual data bit or a bit inserted via bit stuffing. This can be done simply by noticing the maximum number of hi (or low) bits was encountered, and the next bit as to be a 0 (or 1) of bitstuffing. If it isn't, it's likely an error or some kind of flag.

  The main disadvantage of bit-stuffing is that the code rate is unpredictable; it depends on the data being transmitted.